RDF Terms: URIs, Literals, and Blank Nodes

Simon Frost

Overview

Every RDF triple is built from three kinds of *terms*: URIs (resources), Literals (values), and Blank Nodes (anonymous resources). This vignette covers how to create and work with each type.

```
using RDFLib
```

URIs (URIRef)

A URI (Uniform Resource Identifier) uniquely identifies a resource. In RDFLib.jl, they are represented by `URIRef`.

```
# Create URIs directly
person = URIRef("http://example.org/person/alice")
println(person)
println("Type: ", typeof(person))
```

```
URIRef("http://example.org/person/alice")
Type: URIRef
```

```
# Using namespaces (preferred)
ex = Namespace("http://example.org/")
alice = ex("alice")
knows = ex("knows")

println(alice)
println("Value: ", alice.value)
```

```
URIRef("http://example.org/alice")
Value: http://example.org/alice
```

```
# URI utilities
uri = URIRef("http://example.org/vocab#name")
println("Fragment: ", fragment(uri))
println("Defragmented: ", defrag(uri))
println("N3 syntax: ", n3(uri))
```

```
Fragment: name
Defragmented: URIRef("http://example.org/vocab")
N3 syntax: <http://example.org/vocab#name>
```

Literals

Literals represent data values — strings, numbers, dates, booleans, etc. They can have an optional language tag or datatype.

Plain Strings

```
# Simple string literal
name = Literal("Alice")
println(name)
println("Lexical value: ", name.lexical)
println("Datatype: ", datatype(name))
println("N3: ", n3(name))
```

```
Literal("Alice")
Lexical value: Alice
Datatype: nothing
N3: "Alice"
```

Language-Tagged Strings

```
# Literals with language tags
label_en = Literal("Cat"; lang="en")
label_fr = Literal("Chat"; lang="fr")
label_de = Literal("Katze"; lang="de")

println("English: ", label_en, " (lang=", lang(label_en), ")")
println("French:  ", label_fr, " (lang=", lang(label_fr), ")")
println("German:  ", label_de, " (lang=", lang(label_de), ")")
```

```
English: Literal("Cat", lang="en") (lang=en)
French:  Literal("Chat", lang="fr") (lang=fr)
German:  Literal("Katze", lang="de") (lang=de)
```

Typed Literals

```
# Numeric types
age = Literal(30)
println("Integer: ", age, " - datatype: ", datatype(age))

height = Literal(1.75)
println("Float:   ", height, " - datatype: ", datatype(height))

# Boolean
active = Literal(true)
println("Boolean: ", active, " - datatype: ", datatype(active))

# Explicit datatype
score = Literal("98.6"; datatype=XSD.decimal)
println("Decimal: ", score, " - datatype: ", datatype(score))
```

```
Integer: Literal("30", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer")) - datatyp
Float:   Literal("1.75", datatype=URIRef("http://www.w3.org/2001/XMLSchema#double")) - datat
Boolean: Literal("true", datatype=URIRef("http://www.w3.org/2001/XMLSchema#boolean")) - data
Decimal: Literal("98.6", datatype=URIRef("http://www.w3.org/2001/XMLSchema#decimal")) - data
```

XSD Date/Time Types

```
using Dates

# Date and time literals
today_lit = xsd_literal(today())
println("Date: ", n3(today_lit))

now_lit = xsd_literal(now())
println("DateTime: ", n3(now_lit))
```

```
Date: "2026-03-10"^^<http://www.w3.org/2001/XMLSchema#date>
DateTime: "2026-03-10T09:39:56"^^<http://www.w3.org/2001/XMLSchema#dateTime>
```

Extracting Values

```
# convert(Any, lit) extracts the native Julia value from a Literal
println("convert(Any, Literal(42))     = ", convert(Any, Literal(42)), " :: ", typeof(convert
println("convert(Any, Literal(3.14))   = ", convert(Any, Literal(3.14)), " :: ", typeof(conve
println("convert(Any, Literal(true))   = ", convert(Any, Literal(true)), " :: ", typeof(conve
println("convert(Any, Literal(\"hi\")) = ", convert(Any, Literal("hi")), " :: ", typeof(conve
```

```
convert(Any, Literal(42))     = 42 :: Int64
convert(Any, Literal(3.14))   = 3.14 :: Float64
convert(Any, Literal(true))   = true :: Bool
convert(Any, Literal("hi")) = hi :: String
```

Blank Nodes (BNode)

Blank nodes represent anonymous resources — they have no global identifier.

```
# Auto-generated blank node (UUID-based)
b1 = BNode()
println("Auto BNode: ", b1, " (id=", b1.id, ")")

# Named blank node
b2 = BNode("address1")
println("Named BNode: ", b2, " (id=", b2.id, ")")
println("N3 syntax: ", n3(b2))
```

```
Auto BNode: BNode("Nd2ca89ed66a9cd934cd6d38dbdfbe67e") (id=Nd2ca89ed66a9cd934cd6d38dbdfbe67e)
Named BNode: BNode("address1") (id=address1)
N3 syntax: _:address1
```

```
# Blank nodes are useful for structured data without global identity
g = RDFGraph()
ex = Namespace("http://example.org/")

alice = ex("alice")
address = BNode("addr1")

add!(g, Triple(alice, ex("address"), address))
add!(g, Triple(address, ex("street"), Literal("123 Main St")))
add!(g, Triple(address, ex("city"), Literal("Springfield")))
add!(g, Triple(address, ex("zip"), Literal("62701")))

println(serialize(g, TurtleFormat()))
```

```
@prefix ns1: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:alice ns1:address [ ns1:city "Springfield" ;
        ns1:street "123 Main St" ;
        ns1:zip "62701" ] .
```

Variables

Variables are used in SPARQL queries and N3 rules. They represent unknowns to be bound during matching.

```
# Create variables
x = Variable("x")
name_var = Variable("name")

println("Variable: ", x, " - name: ", x.name)
println("N3: ", n3(x))
```

```
Variable: Variable("x") - name: x
N3: ?x
```

Triples

A triple combines a subject, predicate, and object into a single statement.

```
ex = Namespace("http://example.org/")

t = Triple(ex("earth"), RDF.type, ex("Planet"))
println("Subject:   ", t.subject)
println("Predicate: ", t.predicate)
println("Object:    ", t.object)
println("N3:        ", n3(t.subject), " ", n3(t.predicate), " ", n3(t.object), " .")
```

```
Subject:   URIRef("http://example.org/earth")
Predicate: URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type")
Object:    URIRef("http://example.org/Planet")
N3:        <http://example.org/earth> <http://www.w3.org/1999/02/22-rdf-
syntax-ns#type> <http://example.org/Planet> .
```

Type Hierarchy

RDFLib.jl uses Julia's type system to model the RDF term hierarchy:

```
Identifier (abstract)
   Node (abstract)
       IdentifiedNode (abstract)
          URIRef
          BNode
       Formula (N3 quoted graphs)
   Literal
   Variable
```

```
# Type checking
println("URIRef <: Node: ", URIRef <: Node)
println("Literal <: Identifier: ", Literal <: Identifier)
println("BNode <: IdentifiedNode: ", BNode <: IdentifiedNode)
println("Variable <: Identifier: ", Variable <: Identifier)
```

```
URIRef <: Node: true
Literal <: Identifier: true
BNode <: IdentifiedNode: true
Variable <: Identifier: true
```

Equality and Hashing

RDF terms support equality comparison and hashing, which is important for graph operations.

```
# URIs are equal if their values match
println(URIRef("http://example.org/a") == URIRef("http://example.org/a"))  # true
println(URIRef("http://example.org/a") == URIRef("http://example.org/b"))  # false

# Literals compare by value, datatype, and language
println(Literal("hello") == Literal("hello"))     # true
println(Literal(42) == Literal(42))               # true
println(Literal("hi"; lang="en") == Literal("hi"; lang="fr"))  # false
```

```
true
false
true
true
false
```

Parsing Terms from N3 Syntax

You can parse individual terms from N3/SPARQL string syntax:

```
# Parse various term types from N3 notation
println(from_n3("<http://example.org/alice>"))
println(from_n3("\"hello\""))
println(from_n3("\"42\"^^<http://www.w3.org/2001/XMLSchema#integer>"))
println(from_n3("_:blank1"))
```

```
URIRef("http://example.org/alice")
Literal("hello")
Literal("42", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
BNode("blank1")
```